CENTRO FP SUPERIOR
CAMPUS CÁMARA COMERCIO SEVILLA

Laboratorio 1

Ingesta de Fuentes Heterogéneas

Módulo	**Sistemas de Big Data (IABD)**
Sesión	Sesión 5: Procesos de extracción de datos
Profesor	Willman Acosta Lugo
Fecha	17 de noviembre de 2025

Tabla de contenidos

1. Objetivos del Laboratorio **2**
2. Requisitos Previos: El Entorno de Práctica **2**
3. Parte A: Ingesta BATCH (El "Camión" de Datos con Sqoop) **3**
Paso A1: Explorar la Fuente (MySQL) 3
Paso A2: El Comando de Ingesta (Sqoop) 3
Paso A3: Verificar el Destino (HDFS) 4
4. Parte B: Ingesta STREAMING (La "Tubería" de Datos con Flume) **4**
Paso B1: Diseño del Agente (Source, Channel, Sink) 4
Paso B2: Crear el Fichero de Configuración 5
Paso B3: Preparar el Directorio de Ingesta 6
Paso B4: Lanzar el Agente Flume 6
Paso B5: Simular Datos y Verificar 6
5. Discusión y Cierre **7**

1. Objetivos del Laboratorio

Hoy pasamos de la teoría (ETL vs. ELT) a la práctica de ingeniería. El objetivo es enfrentarnos a un escenario real donde debemos ingerir datos de dos fuentes empresariales completamente distintas.

Al finalizar esta sesión, serás capaz de:

- **Implementar** un proceso de ingesta **Batch** desde una BBDD relacional (MySQL) usando **Apache Sqoop**.
- **Configurar** un agente de ingesta **Streaming** para capturar datos en tiempo real (Logs) usando **Apache Flume**.
- **Verificar** la correcta llegada de los datos a nuestro Data Lake (HDFS).
- **Articular** por qué y cuándo se utiliza cada una de estas herramientas fundamentales.
- Utiliza un chatbot como Gemini como herramienta para preparar tu entorno.
- Recuerda que ya tienes tu ecosistema Big Data. Sólo ejecuta Docker.

2. Requisitos Previos: El Entorno de Práctica

Para garantizar que todos partimos del mismo punto y podemos centrarnos en la ingesta (y no en la instalación), trabajaremos sobre nuestro ecosistema ya preparado.

1. **Verificación de Servicios:** Antes de empezar, asegúrate que Hadoop está activo (Deberías ver, como mínimo: NameNode, DataNode, ResourceManager, NodeManager y Jps).
2. Aunque debes "*dokerizar*" un MySQL.

3. Parte A: Ingesta BATCH (El "Camión" de Datos con Sqoop)

Escenario: El departamento de Ventas nos pide migrar su base de datos histórica (ventas_db) a HDFS para que el equipo de analítica pueda cruzarla con otros datos masivos.

Paso A1: Explorar la Fuente (MySQL)

Primero, vamos a "espiar" la base de datos de origen con el usuario que nos han facilitado.

Para esto, genera una base de datos con 1000 registros.

Puedes fijarte en lo que sigue:

```
# Conectamos con el usuario 'sqoopuser' (password 'sqooppassword')
mysql -u sqoopuser -p'sqooppassword'
```

Una vez dentro (mysql>), inspecciona la base de datos:

```
USE ventas_db;
SHOW TABLES;
SELECT * FROM clientes;
SELECT * FROM pedidos;
EXIT;
```

Paso A2: El Comando de Ingesta (Sqoop)

Ahora, ejecutaremos la "magia" de Sqoop. Este comando le dice a Sqoop que use el driver de MySQL para conectarse, autenticarse y mapear la tabla pedidos en un trabajo MapReduce que depositará los datos en HDFS.

```
sqoop import \
--connect jdbc:mysql://localhost/ventas_db \
--username sqoopuser \
--password 'sqooppassword' \
--table pedidos \
--target-dir /user/iabd_admin/ingesta/ventas/pedidos \
--m 1
```

*(**Nota**: El \ al final de cada línea es solo para formatear el comando. Puedes escribirlo en una sola línea. El --m 1 fuerza a usar un solo mapper para no sobrecargar nuestra VM).*

Paso A3: Verificar el Destino (HDFS)

Si el comando sqoop *import* finaliza con éxito (verás un resumen de MapReduce), los datos ya no están solo en MySQL, sino también en nuestro Data Lake.

1. Comprueba que el directorio se ha creado:

```
hdfs dfs -ls /user/iabd_admin/ingesta/ventas
```

3. (Deberías ver la carpeta pedidos).
4. Mira los ficheros que Sqoop ha creado dentro:

```
hdfs dfs -ls /user/iabd_admin/ingesta/ventas/pedidos
```

6. (Verás un _SUCCESS y uno o más ficheros part-m-00000...).
7. Lee los datos directamente desde HDFS:

```
hdfs dfs -cat
/user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
```

9. (¡Deberías ver los datos de tu tabla pedidos en formato CSV!).

¡Felicidades! Has completado tu primera ingesta Batch.

4. Parte B: Ingesta STREAMING (La "Tubería" de Datos con Flume)

Escenario: El departamento de IT nos pide capturar *en tiempo real* los logs de acceso al servidor web. Los logs se escriben constantemente en un directorio y deben ser enviados a HDFS al instante para su monitorización.

Paso B1: Diseño del Agente (Source, Channel, Sink)

Usaremos un agente Flume con tres componentes:

1. **Source (Fuente)**: Un spooldir (Directorio "spooling"). Flume monitorizará una carpeta. Cualquier fichero cerrado que aparezca ahí, será ingerido.
2. **Channel (Canal)**: Un memory channel. Un búfer de alta velocidad en RAM.
3. **Sink (Destino)**: Un hdfs sink, que escribirá los datos en nuestro Data Lake.

Paso B2: Crear el Fichero de Configuración

Flume se configura con un fichero .conf. Vamos a crear el nuestro.

1. Crea un directorio para nuestras configuraciones:

```
mkdir -p ~/flume_configs
cd ~/flume_configs
```

2. Crea el fichero de configuración:

```
nano weblogs.conf
```

3. **Copia y pega** el siguiente contenido exacto dentro de nano:

```
# --- AGENTE: "agent" ---
# Nombramos los componentes
agent.sources = s1
agent.channels = c1
agent.sinks = k1

# --- SOURCE (s1): SpoolDir ---
# Vigila un directorio en busca de ficheros
agent.sources.s1.type = spooldir
agent.sources.s1.spoolDir = /home/iabd_admin/flume_ingest
agent.sources.s1.fileHeader = true
agent.sources.s1.channels = c1

# --- CHANNEL (c1): Memory ---
# Un buffer en RAM
agent.channels.c1.type = memory
agent.channels.c1.capacity = 1000
agent.channels.c1.transactionCapacity = 100

# --- SINK (k1): HDFS ---
# Escribe los datos en HDFS
agent.sinks.k1.type = hdfs
agent.sinks.k1.hdfs.path =
hdfs://localhost:9000/user/iabd_admin/ingesta/weblogs/
agent.sinks.k1.hdfs.fileType = DataStream
agent.sinks.k1.hdfs.writeFormat = Text
agent.sinks.k1.hdfs.rollInterval = 60
```

```
agent.sinks.k1.hdfs.rollSize = 0
agent.sinks.k1.hdfs.rollCount = 0
agent.sinks.k1.hdfs.useLocalTimeStamp = true
agent.sinks.k1.channel = c1
```

4. Guarda (Ctrl+O, Enter) y Sal (Ctrl+X) de nano.

Paso B3: Preparar el Directorio de Ingesta

El spooldir que hemos definido (/home/iabd_admin/flume_ingest) debe existir.

```
mkdir /home/iabd_admin/flume_ingest
```

Paso B4: Lanzar el Agente Flume

(¡Importante! Necesitarás **dos terminales** SSH conectadas a tu VM).

1. **En la Terminal 1:** Lanza el agente Flume. Se quedará "escuchando" indefinidamente.

```
flume-ng agent \
--conf ~/flume_configs/ \
--conf-file ~/flume_configs/weblogs.conf \
--name agent \
-Dflume.root.logger=INFO,console
```

Paso B5: Simular Datos y Verificar

1. **En la Terminal 2:** Vamos a simular que el servidor web genera logs. Crearemos un fichero y lo moveremos al directorio que Flume está vigilando.

```
# 1. Creamos un fichero de log de ejemplo
echo "192.168.1.1 - GET /index.html" > ~/log_temp.txt
echo "192.168.1.5 - POST /login.php" >> ~/log_temp.txt

# 2. Lo movemos al spool (Flume lo detectará al instante)
mv ~/log_temp.txt /home/iabd_admin/flume_ingest/log_file_01.txt
```

2. **Observa la Terminal 1:** Verás la actividad de Flume, detectando el fichero y enviándolo al Sink.

3. **Verifica en HDFS (Terminal 2):**

```
hdfs dfs -ls /user/iabd_admin/ingesta/weblogs
```

4. (Verás el fichero .tmp que está escribiendo Flume, o el fichero ya cerrado).

5. **Genera más datos:** Repite el paso de mv con otro fichero (log_file_02.txt). Verás cómo Flume ingiere de nuevo.

¡Felicidades! Has completado una ingesta en Streaming.

5. Discusión y Cierre

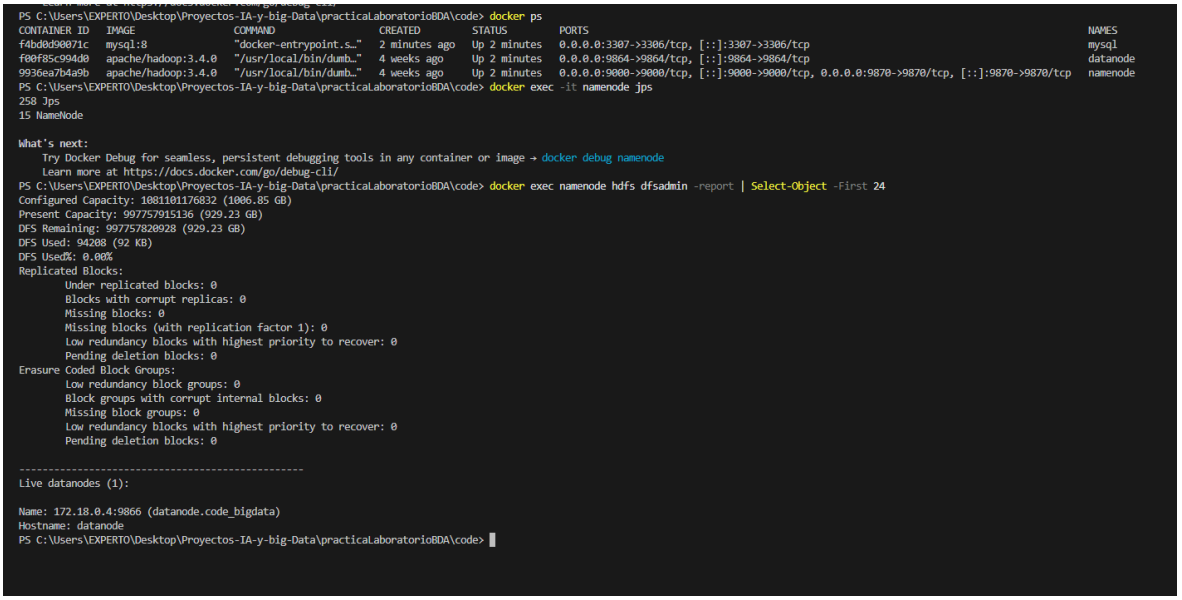
- **¿Por qué no usamos Sqoop para los logs? ¿Qué pasaría?**
  Sqoop está diseñado para transferir datos entre una base de datos relacional y Hadoop vía JDBC, y funciona por lotes. Para logs no tiene una fuente de datos que leer, ya que no son tablas. Además, no es continuo: habría que relanzarlo manualmente o con un planificador, y los logs no llegarían "al instante" como pide el escenario de IT.

- **¿Por qué no usamos Flume para la base de datos? ¿Se podría?**
  Flume está pensado para recoger flujos de eventos (logs, mensajes) y moverlos continuamente. No sabe leer tablas, particionar una consulta ni paralelizar con varios mappers como Sqoop. Se podría forzar con un source personalizado que haga consultas periódicas, pero es mucho más trabajo y menos fiable que usar Sqoop, que lo hace de forma nativa.

- **En Flume usamos un `memory channel`: ¿qué riesgo tiene en producción?**
  Los eventos que están en el canal solo existen en RAM. Si el agente se cae, el servidor se reinicia o el sink no puede escribir y el búfer se llena, **se pierden datos sin posibilidad de recuperarlos**. Para producción se usa un *file channel* (persistente en disco, más lento pero seguro) o se pone una cola como Kafka entre medias.

EVIDENCIAS DE LA PRÁCTICA

Evidencia 0: Entorno (Docker y HDFS)



Los servicios NameNode y DataNode están activos en contenedores Docker separados (`namenode` y `datanode`), y el DataNode aparece como activo (`Live datanodes (1)`) en el informe de HDFS, lo que confirma que ambos se comunican correctamente. Junto a ellos está levantado el contenedor `mysql`, que actúa como fuente de datos. En este entorno no se despliega YARN (ResourceManager/NodeManager), por lo que Sqoop ejecuta el trabajo MapReduce en modo local.

Parte A: Ingesta BATCH (El "Camión" de Datos con Sqoop)

Evidencia A1: Fuente de datos en MySQL

```
PS C:\Users\EXPERTO\Desktop\Proyectos-IA-y-big-Data\practicaLaboratorioBDA\code> docker exec -it mysql mysql -u sqoopuser -p'sqooppassword'
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.13 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE ventas_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+--------------------+
| Tables_in_ventas_db |
+--------------------+
| clientes           |
| pedidos            |
+--------------------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) FROM pedidos;
+----------+
| COUNT(*) |
+----------+
|     1000 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) FROM clientes;
+----------+
| COUNT(*) |
+----------+
|      100 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT * FROM clientes LIMIT 5;
+----+----------+-----------+
| id | nombre   | ciudad    |
+----+----------+-----------+
|  1 | Cliente1 | Barcelona |
|  2 | Cliente2 | Valencia  |
|  3 | Cliente3 | Sevilla   |
|  4 | Cliente4 | Madrid    |
|  5 | Cliente5 | Barcelona |
+----+----------+-----------+
5 rows in set (0.00 sec)

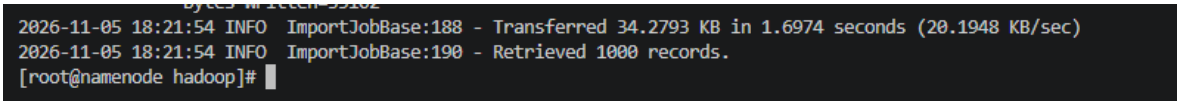
mysql> SELECT * FROM pedidos LIMIT 5;
+----+------------+-----------+---------+------------+
| id | cliente_id | producto  | importe | fecha      |
+----+------------+-----------+---------+------------+
|  1 |          2 | Producto1 |  360.64 | 2025-01-02 |
|  2 |          3 | Producto2 |   16.64 | 2025-01-03 |
|  3 |          4 | Producto3 |    1.27 | 2025-01-04 |
|  4 |          5 | Producto4 |  456.43 | 2025-01-05 |
|  5 |          6 | Producto5 |  278.33 | 2025-01-06 |
+----+------------+-----------+---------+------------+
5 rows in set (0.00 sec)

mysql> EXIT;
```

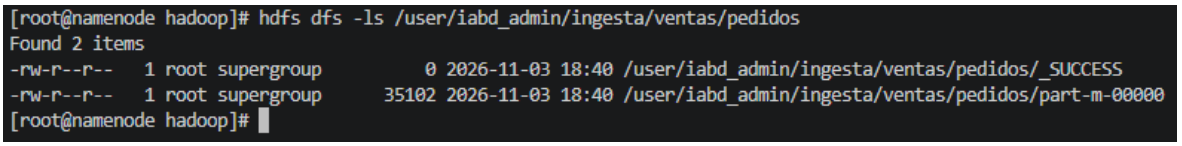
Conexión con el usuario `sqoopuser` a la base de datos `ventas_db`, alojada en el contenedor `mysql`. Se comprueba que existen las tablas `clientes` (100 registros) y `pedidos` (1000 registros), que será la tabla a importar a HDFS con Sqoop.

Evidencia A2: Importación con Sqoop

sqoop import --connect jdbc:mysql://mysql:3306/ventas_db --username sqoopuser --password 'sqooppassword' --table pedidos --columns "id,cliente_id,producto,importe,fecha" --target-dir /user/iabd_admin/ingesta/ventas/pedidos --m 1



Evidencia 1: Archivo de éxito y partición de datos



En la captura debe verse un archivo llamado `_SUCCESS` (que confirma que el proceso MapReduce terminó sin errores) y un archivo llamado `part-m-00000` (que contiene los datos exportados por el único mapper que configuramos).

Evidencia 2: Lectura de los datos en HDFS

```
^C[root@namenode hadoop]# hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/pa-m-00000
1,2,Producto1,360.64,2025-01-02
2,3,Producto2,16.64,2025-01-03
3,4,Producto3,1.27,2025-01-04
4,5,Producto4,456.43,2025-01-05
5,6,Producto5,278.33,2025-01-06
6,7,Producto6,22.38,2025-01-07
7,8,Producto7,276.89,2025-01-08
8,9,Producto8,317.34,2025-01-09
9,10,Producto9,256.01,2025-01-10
10,11,Producto10,328.03,2025-01-11
11,12,Producto11,372.12,2025-01-12
12,13,Producto12,376.49,2025-01-13
13,14,Producto13,266.11,2025-01-14
14,15,Producto14,201.10,2025-01-15
15,16,Producto15,207.14,2025-01-16
16,17,Producto16,432.42,2025-01-17
17,18,Producto17,40.66,2025-01-18
18,19,Producto18,406.07,2025-01-19
19,20,Producto19,408.35,2025-01-20
20,21,Producto0,323.53,2025-01-21
21,22,Producto1,392.59,2025-01-22
22,23,Producto2,492.37,2025-01-23
23,24,Producto3,284.09,2025-01-24
24,25,Producto4,443.35,2025-01-25
25,26,Producto5,364.46,2025-01-26
26,27,Producto6,492.25,2025-01-27
27,28,Producto7,367.89,2025-01-28
28,29,Producto8,362.70,2025-01-29
29,30,Producto9,209.82,2025-01-30
30,31,Producto10,461.01,2025-01-31
31,32,Producto11,175.58,2025-02-01
32,33,Producto12,494.86,2025-02-02
33,34,Producto13,447.57,2025-02-03
34,35,Producto14,253.28,2025-02-04
35,36,Producto15,423.67,2025-02-05
36,37,Producto16,358.52,2025-02-06
37,38,Producto17,21.57,2025-02-07
38,39,Producto18,32.31,2025-02-08
39,40,Producto19,96.81,2025-02-09
40,41,Producto0,387.16,2025-02-10
41,42,Producto1,145.34,2025-02-11
42,43,Producto2,65.24,2025-02-12
43,44,Producto3,390.17,2025-02-13
44,45,Producto4,255.15,2025-02-14
45,46,Producto5,105.21,2025-02-15
46,47,Producto6,260.63,2025-02-16
47,48,Producto7,487.49,2025-02-17
48,49,Producto8,155.56,2025-02-18
49,50,Producto9,315.34,2025-02-19
50,51,Producto10,110.03,2025-02-20
```

Esta captura muestra el interior del archivo creado, donde se verán los registros de la tabla `pedidos` (números, productos, fechas, etc.) en formato de texto plano separado por comas (CSV), confirmando que la extracción de la base de datos relacional se realizó con integridad.

Parte B: Ingesta STREAMING (La "Tubería" de Datos con Flume)

Evidencia B1: Fichero de configuración weblogs.conf

```
[root@namenode hadoop]# ls ~/flume_configs
weblogs.conf
[root@namenode hadoop]# cat ~/flume_configs/weblogs.conf
# --- AGENTE: "agent" ---
agent.sources = s1
agent.channels = c1
agent.sinks = k1

# --- SOURCE (s1): SpoolDir ---
agent.sources.s1.type = spooldir
agent.sources.s1.spoolDir = /home/iabd_admin/flume_ingest
agent.sources.s1.fileHeader = true
agent.sources.s1.channels = c1

# --- CHANNEL (c1): Memory ---
agent.channels.c1.type = memory
agent.channels.c1.capacity = 1000
agent.channels.c1.transactionCapacity = 100

# --- SINK (k1): HDFS ---
agent.sinks.k1.type = hdfs
agent.sinks.k1.hdfs.path = hdfs://namenode:9000/user/iabd_admin/ingesta/weblogs/
agent.sinks.k1.hdfs.fileType = DataStream
agent.sinks.k1.hdfs.writeFormat = Text
agent.sinks.k1.hdfs.rollInterval = 60
agent.sinks.k1.hdfs.rollSize = 0
agent.sinks.k1.hdfs.rollCount = 0
agent.sinks.k1.hdfs.useLocalTimeStamp = true
agent.sinks.k1.channel = c1
[root@namenode hadoop]#
```

Contenido del fichero de configuración del agente Flume. Se definen tres componentes: un source de tipo spooldir (s1) que vigila el directorio /home/iabd_admin/flume_ingest, un channel de tipo memory (c1) con capacidad de 1000 eventos que actúa como búfer en RAM, y un sink de tipo hdfs (k1) que escribe en hdfs://localhost:9000/user/iabd_admin/ingesta/weblogs/ cerrando un fichero cada 60 segundos.

Evidencia B2: Directorio de ingesta (spooldir)

```
[root@namenode hadoop]# ls -ld /home/iabd_admin/flume_ingest
drwxr-xr-x 3 root root 4096 Nov  3 18:50 /home/iabd_admin/flume_ingest
[root@namenode hadoop]#
```

Se comprueba que existe el directorio /home/iabd_admin/flume_ingest, que es el que vigila el source de tipo spooldir. Este directorio debe existir antes de lanzar el agente, ya que de lo contrario Flume no puede arrancar. Cualquier fichero que se mueva aquí será detectado e ingerido por el agente.

1. Comprobar el procesamiento local

```
[root@namenode hadoop]# ls -l /home/iabd_admin/flume_ingest
total 8
-rw-r--r-- 1 root root 60 Nov  3 18:49 log_file_01.txt.COMPLETED
-rw-r--r-- 1 root root 86 Nov  3 18:50 log_file_02.txt.COMPLETED
[root@namenode hadoop]#
```

Confirma que el agente de Flume detectó los archivos en el directorio de origen (spooldir) y los procesó correctamente, añadiendoles la extensión `.COMPLETED`.

2. Verificar la llegada al Data Lake (HDFS)

```
[root@namenode hadoop]# hdfs dfs -ls /user/iabd_admin/ingesta/weblogs
Found 2 items
-rw-r--r--   1 root supergroup         60 2026-11-03 18:50 /user/iabd_admin/ingesta/weblogs/FlumeData.1793731746218
-rw-r--r--   1 root supergroup         86 2026-11-03 18:51 /user/iabd_admin/ingesta/weblogs/FlumeData.1793731828376
[root@namenode hadoop]#
```

Verifica que los archivos se han escrito en el sistema de archivos distribuido bajo la ruta configurada en el *Sink*, generando archivos con el prefijo `FlumeData`.

3. Validar el contenido ingerido

```
[root@namenode hadoop]# hdfs dfs -cat /user/iabd_admin/ingesta/weblogs/FlumeData.*
192.168.1.1 - GET /index.html
192.168.1.5 - POST /login.php
10.0.0.7 - GET /productos.html
10.0.0.9 - GET /carrito.html
10.0.0.7 - POST /pago.php
[root@namenode hadoop]#
```

Imprime el texto directamente desde HDFS, probando que el contenido (los logs simulados con direcciones IP y peticiones GET/POST) se transfirió de forma íntegra y sin pérdida de datos.